

Aula 12 — Quando um colaborador começa a variar

Como evoluir uma colaboração quando o objeto que realiza o trabalho pode variar?

DO PROJETO QUE JÁ FUNCIONA

Sabemos acrescentar regras

- Aula 09: `Pedido` protege o fechamento.
- Aula 10: localiza linhas; o item protege sua quantidade.
- Aula 11: o raciocínio funciona em outro domínio.

Agora chegou uma responsabilidade que ainda não existia.

NOVO REQUISITO

O pedido precisa de uma forma de entrega

A escolha acontece **no fechamento**.

Por enquanto, só existe **entrega normal**.

Custo fixo: **R\$ 10**.

Quem deveria conhecer essa regra?

UMA RESPONSABILIDADE PRÓPRIA

Um objeto pode realizar esse trabalho

```
public class EntregaNormal {  
    public double calcularCusto(Pedido pedido) {  
        return 10.0;  
    }  
}
```

Recebe o pedido como contexto. Nesta regra fixa, não precisa consultar seus dados.

Pedido mantém a referência necessária

```
private EntregaNormal entrega;

public void fechar(EntregaNormal entrega) {
    if (!fechado && entrega != null) {
        this.entrega = entrega;
        fechado = true;
    }
}
```

O pedido nasce aberto. A entrega é escolhida no fechamento.

SOLICITAR O COMPORTAMENTO

Pedido coordena o cálculo da entrega

```
public double calcularCustoEntrega() {  
    return entrega.calcularCusto(this);  
}
```

Pedido precisa saber que a regra devolve 10.0 ?

Java em foco — o próprio pedido

```
return entrega.calcularCusto(this);
```

`this` referencia o `Pedido` que executa o método.

O colaborador recebe esse mesmo pedido como argumento.

Nenhuma cópia é criada.

A NOVA DEPENDÊNCIA

Uma entrega, uma colaboração

Pedido



EntregaNormal

```
private EntregaNormal entrega;
```

Visão simplificada: a referência mantida no campo permite alcançar a entrega.

ATIVIDADE

EXPLORE ANTES DE EVOLUIR

A distribuição faz sentido?

1. Quem conhece a fórmula do frete?
2. Se ela mudar para R\$ 12, onde esperamos alterar código?
3. Essa solução é adequada enquanto existe só entrega normal?

O QUE JÁ CONSEGUIMOS PRESERVAR

A regra pertence à entrega

`EntregaNormal` conhece o custo e pode mudar sua regra.

`Pedido` continua solicitando o mesmo cálculo.

A solução é adequada ao requisito atual.

Dois resultados, duas responsabilidades

```
// teclado: 150.0; mouse: 80.0  
Pedido pedido = new Pedido();  
pedido.adicionarItem(teclado, 2);  
pedido.adicionarItem(mouse, 1);  
pedido.fechar(new EntregaNormal());  
System.out.println(pedido.calcularTotal());  
System.out.println(pedido.calcularCustoEntrega());
```

Quais valores serão impressos? De onde vem cada um?

O SISTEMA CONTINUA FUNCIONANDO

Itens e entrega têm cálculos separados

380.0

10.0

`calcularTotal()` continua somando subtotais dos itens.

`calcularCustoEntrega()` solicita a nova colaboração.

O REQUISITO EVOLUIU

Alguns pedidos precisam de entrega expressa

```
public class EntregaExpressa {  
    public double calcularCusto(Pedido pedido) {  
        return 25.0;  
    }  
}
```

Também calcula o custo da entrega. Usa outra regra.

ATIVIDADE

UMA PREVISÃO DE COMPILAÇÃO

Podemos fornecer a nova entrega?

```
// fechar recebe EntregaNormal.  
EntregaExpressa expressa = new EntregaExpressa();  
Pedido urgente = new Pedido();  
urgente.fechar(expressa);
```

O compilador aceita essa chamada? Por quê?

ONDE ESTÁ O LIMITE?

O parâmetro pede uma classe concreta

```
public void fechar(EntregaNormal entrega)
```

O argumento é `EntregaExpressa` .

No código mostrado, esse tipo não é aceito como `EntregaNormal` .

Ter `calcularCusto(Pedido)` nas duas classes não basta.

O QUE MUDOU PARA O PEDIDO?

Ainda precisamos do mesmo comportamento

Pedido continua precisando **calcular o custo da entrega**.

Agora existem duas classes capazes de realizar esse trabalho.

A responsabilidade mudou ou mudou quem pode realizá-la?

Trocar objetos não é sempre trocar classes

Na missão:

```
Sensor temperatura = new Sensor("Temperatura", "°C");  
Sensor umidade = new Sensor("Umidade", "%");
```

Dois objetos da mesma classe.

Agora: `EntregaNormal` e `EntregaExpressa` são classes diferentes.

ATIVIDADE

NORMAL E EXPRESSA PRECISAM COEXISTIR

Como guardar a entrega escolhida?

Cada pedido recebe **uma escolha no fechamento**:
normal custa R\$ 10; expressa custa R\$ 25.

Com os recursos que já conhecemos:

1. Que referências permitiriam alcançar os dois colaboradores?
2. Como registrar qual deles atende a este pedido?

Dois campos permitem alcançar os colaboradores

```
private EntregaNormal entregaNormal;  
private EntregaExpressa entregaExpressa;
```

Suponha que os dois campos já apontem para objetos existentes.

Ainda falta registrar **qual entrega foi escolhida para este pedido**.

Acrescentamos um indicador aos dois campos

```
private int tipoEntrega;
```

No fechamento, esta hipótese guarda um código válido:

- 1 : usar entregaNormal ;
- 2 : usar entregaExpressa .

Agora podemos decidir a qual objeto solicitar o custo.

A escolha guardada determina a chamada

```
public double calcularCustoEntrega() {  
    if (tipoEntrega == 1) {  
        return entregaNormal.calcularCusto(this);  
    } else {  
        return entregaExpressa.calcularCusto(this);  
    }  
}
```

Recorte para pedidos fechados: código 1 ou 2, colaboradores disponíveis.

ATIVIDADE

EXECUTE MENTALMENTE A HIPÓTESE

Dois pedidos, duas escolhas

Pedido fechado	Escolha guardada
normal	tipoEntrega = 1
urgente	tipoEntrega = 2

```
System.out.println(normal.calcularCustoEntrega());  
System.out.println(urgente.calcularCustoEntrega());
```

Quais valores aparecem? Qual classe decide **quem** fará cada cálculo?

A HIPÓTESE ATENDE ÀS DUAS OPÇÕES

Funciona, mas Pedido conhece cada alternativa

```
10.0  
25.0
```

Pedido conhece os dois tipos, os códigos 1 e 2 e a seleção.

As entregas continuam responsáveis pelos valores de cada custo.

O REQUISITO MUDA OUTRA VEZ

Agora também podemos retirar no local

```
public class RetiradaLocal {  
    public double calcularCusto(Pedido pedido) {  
        return 0.0;  
    }  
}
```

O cliente quer escolher retirada no fechamento do pedido.

ATIVIDADE

USE A MESMA HIPÓTESE PARA INVESTIGAR O IMPACTO

Criar RetiradaLocal.java é suficiente?

Pedido ainda tem dois campos e seleciona entre os códigos 1 e 2.

1. Onde guardaríamos a referência para a retirada?
2. Como representaríamos essa terceira escolha?
3. Qual método precisaria solicitar o custo ao novo objeto?

Pedido ganha conhecimento da terceira opção

```
private RetiradaLocal retirada;
```

Além dos dois colaboradores anteriores, precisamos fornecer a retirada.

A convenção passa a ter três códigos: **1** normal, **2** expressa, **3** retirada.

O fechamento e o código que monta o pedido também precisam aceitar essa escolha.

O mesmo método precisa de outro caminho

```
public double calcularCustoEntrega() {  
    if (tipoEntrega == 1) {  
        return entregaNormal.calcularCusto(this);  
    } else if (tipoEntrega == 2) {  
        return entregaExpressa.calcularCusto(this);  
    } else {  
        return retirada.calcularCusto(this);  
    }  
}
```

Recorte: código 1, 2 ou 3 ; colaboradores já disponíveis.

CONCEITO-CHAVE

O QUE A TERCEIRA OPÇÃO REVELOU

Conceito-chave — ponto de variação

O trabalho solicitado continua sendo **calcular o custo da entrega**.

Quem realiza esse trabalho pode variar:

`EntregaNormal` , `EntregaExpressa` ou `RetiradaLocal` .

Esse trabalho com alternativas é o ponto de variação que encontramos.

VOLTE À SOLUÇÃO INICIAL — UMA ENTREGA NORMAL

O campo inicial limita quem pode colaborar

```
private EntregaNormal entrega;  
  
public void fechar(EntregaNormal entrega) {  
    // guarda a escolha no primeiro fechamento válido  
}
```

O tipo pede `EntregaNormal`, embora o pedido precise apenas de alguém capaz de calcular o custo.

CONCEITO-CHAVE

NOMEAR A DEPENDÊNCIA QUE OBSERVAMOS

Conceito-chave — acoplamento

`Pedido` depende de `EntregaNormal` para realizar seu trabalho.

O campo e o parâmetro de `fechar` nomeiam essa classe concreta.

Essa dependência atendia ao requisito inicial.

A entrega expressa revelou seu limite.

ARMADILHA

COMPARE O MOTIVO DE CADA CONDIÇÃO

Qual mudança faz cada trecho crescer?

Trecho em Pedido	O que ele decide
<code>if (!fechado)</code>	se o pedido permite uma edição
<code>if (tipoEntrega == 1)</code>	qual classe de entrega será chamada

Foi a **nova entrega** que exigiu ampliar a segunda seleção.

O diagnóstico depende da responsabilidade que cada condição expressa.

NA SOLUÇÃO INICIAL, MUDE SOMENTE QUEM CRIA O OBJETO

Criar a expressa em Main resolve o encaixe?

```
EntregaExpressa expressa = new EntregaExpressa();  
Pedido pedido = new Pedido();  
pedido.fechar(expressa);
```

`fechar` ainda recebe `EntregaNormal`. Esta chamada continua sem compilar.
Precisamos representar em `Pedido` a capacidade comum às entregas.

UMA SEGUNDA PREOCUPAÇÃO

Vamos mudar uma colaboração que funciona

Mesmo que a entrega expressa passe a funcionar...

Como saber se preservamos o total, as edições permitidas e o fechamento?

Que verificações do Projeto 1 precisamos repetir?

Registre os resultados esperados

```
pedido.adicionarItem(teclado, 2);  
pedido.adicionarItem(mouse, 1);  
System.out.println(pedido.calcularTotal());  
pedido.removerItem(mouse);  
System.out.println(pedido.calcularTotal());  
pedido.alterarQuantidade(teclado, 3);  
System.out.println(pedido.calcularTotal());
```

Teclado: R\$ 150; mouse: R\$ 80. Quais três valores são esperados?

CONTINUE O MESMO CENÁRIO

E depois de fechar?

```
pedido.fechar(new EntregaNormal());  
pedido.adicionarItem(mouse, 1);  
pedido.removerItem(teclado);  
pedido.alterarQuantidade(teclado, 1);  
  
System.out.println(pedido.calcularTotal());  
System.out.println(pedido.calcularCustoEntrega());
```

Quais valores devem continuar corretos?

EVIDÊNCIAS QUE PODEMOS REPETIR

O comportamento esperado está explícito

```
380.0 → 300.0 → 450.0 → 450.0  
Entrega normal: 10.0
```

Também precisamos verificar zero, negativo e inclusão repetida.

Repetir e automatizar essas verificações prepara a próxima aula.

Projeto 2 começa com lembretes

Nas aulas, continuamos com `Pedido`. Nos laboratórios, evoluímos um sistema de lembretes e notificações.

1. evoluir o canal de e-mail existente;
2. permitir também SMS;
3. acrescentar um painel local.

Onde o código precisa mudar a cada nova alternativa?

SÍNTESE

DA UNIDADE 01 À UNIDADE 02

Até aqui, quem realiza cada responsabilidade

- Unidade 01: objetos colaboram.
- Aula 12: o colaborador pode ser de outra classe.
- Queremos depender do trabalho necessário sem espalhar as alternativas.
- Antes de mudar a estrutura, precisamos verificar o que já funciona.

A PRÓXIMA MUDANÇA PRECISA PRESERVAR O COMPORTAMENTO

Antes de mudar Pedido, como saber se ainda funciona?

```
private EntregaNormal entrega;
```

Queremos mudar essa dependência para aceitar outras entregas.

Como saber se a mudança que fizemos preservou o total, as edições e o fechamento?